

Sesión 1 de Ako — resumen técnico

Sesión del 28 y 29 de septiembre de 2026. Objetivo del spec: proyecto vacío en Android Studio dentro de `C:\AKO\app-ako`, primer arranque en el emulador, Git y repositorio público en GitHub, primer commit. Estado: **cerrada** (commit de código `a047fb6`, ficha `1d620b0`).

1. Visión general: la cadena de herramientas

- **Android Studio** (2026.1.3, Quail 3): el IDE. No se actualiza hasta la entrega (P19).
- **Android SDK** en `C:\Android\Sdk`: plataformas de API (34, 36, 37), `build-tools`, `platform-tools` (con `adb`) y el emulador. La ruta la apunta `local.properties` (`sdk.dir`), que no se sube a Git porque depende del PC.
- **Emulador** `Pixel_6_API_34`: dispositivo virtual x86_64 con aceleración AEHD. Se abre dentro de Android Studio, en el panel `*Running Devices*`.
- **Gradle** 9.6.0 con **AGP 9.3.3** (Android Gradle Plugin): el sistema de construcción. Compila Kotlin, procesa recursos, ejecuta KSP, empaqueta la APK y lanza las pruebas. La versión de Gradle la fija el `*wrapper*` (`gradle-wrapper.properties`), así compila igual en cualquier PC.
- **Git** (repositorio en `C:\AKO`, rama `main`) y **GitHub** (`dhuaniel-dot/ako`, público).

2. El proyecto

- Plantilla **Empty Views Activity** (vistas XML; la `*Empty Activity*` es la de Compose, que el spec excluye).
- `namespace` y `applicationId` = `yunkang.ako` (P104; sustituye a `es.daniel.ako` del spec). El `applicationId` identifica la app de forma única en el dispositivo y en las tiendas: si cambia, Android la trata como otra app. El `namespace` es el paquete de la clase `R` y del código.
- `minSdk 26`: `SecretKeyFactory.getInstance("PBKDF2WithHmacSHA256")`, necesario para guardar el PIN como sal + hash (regla del spec: nunca en claro), está disponible desde la API 26; así no hace falta ninguna librería de criptografía externa (el spec las prohíbe). Cobertura según el asistente: ~98,4 % de los dispositivos.
- `compileSdk` y `targetSdk 37`: se compila contra la API más reciente (Glide 5.0.9 la exige) y se declara que la app está probada para ella. `minSdk` es el suelo; `targetSdk`, el comportamiento para el que está pensada.
- `app_name` = **AKO** (P105).

3. Configuración de Gradle (del `stack-verificado`, P106: copiada por Claude)

- `gradle/libs.versions.toml` — el **catálogo de versiones**: bloques `[versions]`, `[libraries]` (alias → coordenadas Maven + `version.ref`) y `[plugins]`. Un único sitio para las versiones de las

dependencias; no tiene que ver con la versión de la app (`versionCode` / `versionName` , en `app/build.gradle.kts`).

- **app/build.gradle.kts** — plugins `com.android.application` y `com.google.devtools.ksp` ; `compileSdk` / `minSdk` / `targetSdk` ; `testInstrumentationRunner` ; `buildFeatures { viewBinding = true }` ; Java 17; dependencias:
 - base de AndroidX: `core-ktx` , `appcompat` , `material` , `constraintlayout` , `recyclerview` , `activity-ktx` , `fragment-ktx` ;
 - MVVM: `lifecycle-viewmodel-ktx` , `lifecycle-livedata-ktx` , `kotlinx-coroutines-android` ;
 - **Room** 2.8.5 (`room-runtime` , `room-ktx` , `ksp(room-compiler)`) y **Glide** 5.0.9 (`glide` , `ksp(glide ksp)`);
 - pruebas: `junit` (`test/`), `room-testing` , `androidx.test.ext:junit` y `runner` (`androidTest/`).
- **build.gradle.kts** raíz: declara los plugins con `apply false` .
- **settings.gradle.kts** : repositorios (Google, Maven Central), `rootProject.name = "AKO"` , módulo `:app` .
- **gradle.properties** : `android.useAndroidX` , `kotlin.code.style=official` , `android.nonTransitiveRClass` .
- **KSP** (Kotlin Symbol Processing) genera código a partir de anotaciones (`@Entity` , `@Dao` , `@GlideModule`). Por eso existe `imagenes/AkoGlideModule.kt` (una clase `AppGlideModule` vacía): sin ella Glide con KSP no compila. Prueba de que funciona: `GeneratedAppGlideModuleImpl` en `java (generated)` .
- **Kotlin 2.2.10** va integrado en AGP 9: no se declara el plugin `org.jetbrains.kotlin.android` .
- Verificación: `assembleDebug` y `testDebugUnitTest` en verde; `*Sync*` y `*Run*` correctos en Android Studio.

4. Git y GitHub

- `git init -b main` en `C:\AKO` (nunca dentro de `app-ako/` : un `.git` anidado dejaría la carpeta vacía en GitHub).
- **.gitignore** : fuera `build/` , `.gradle/` , `.idea/` , `local.properties` , APKs, keystores, `.env` , `.mcp.json` , `claude_desktop_config.json` , `.claude/settings.local.json` .
- **Commit** = instantánea del estado del repositorio con autor, fecha y mensaje. **Push** = enviar los commits locales al remoto (`origin` = GitHub). Creación del remoto con `gh repo create ako --public --source=. --push` .
- **Hook** `comprobar-commit.sh` (PreToolUse de Claude Code): exige el formato `S<N>:` , la ficha de la sesión, ninguna clave de API en lo cambiado y, desde el 29 sep, que el autor sea Daniel con el correo anónimo de GitHub.
- **README.md**: la portada del repositorio (qué es, stack, cómo abrir, cómo probar, estructura, uso de IA).

5. Uso de IA y decisiones de trabajo

- **Consulta al profesor** (29 sep): el uso de Claude Code está permitido; lo malo sería que la IA diseñara y ejecutara el proyecto sola o depender de ella; puede escribir las partes repetitivas (p. ej. datos de la base de datos).
- **P111**: desde el 29 sep, los commits llevan `Co-Authored-By: Claude` (ajuste `attribution` en `.claude/settings.json`). Sustituye a P226.
- **P112**: las decisiones que cambian cómo funciona el código las elige Daniel entre **mínimo 3 opciones**, al menos una de Claude y otra de una fuente de internet con enlace.
- **P108 / seguridad**: sin correos ni datos personales en los documentos; revisión de todo el historial buscando claves (limpio); commits con el correo anónimo de GitHub. Los 3 commits del otro chat firmados como «Claude» siguen en el historial (no se reescribe: `push --force` prohibido).

6. Pendiente de comprender (se repasa al abrir la S2)

- **Gradle**: qué hace en cada compilación (configuración → tareas: KSP, compilar Kotlin, recursos, empaquetar, pruebas).
- **Catálogo de versiones**: versiones de las dependencias, no del proyecto.
- `minSdk 26`: disponibilidad de `PBKDF2WithHmacSHA256` desde la API 26.

7. Para defender en el vídeo

- Por qué el proyecto está en `C:\AKO\app-ako`, junto a `docs/`: una sola raíz de repositorio; la documentación viaja con el código.
- Qué hace Gradle y qué es el catálogo de versiones.
- Por qué `minSdk 26` y qué es `targetSdk`.
- Qué es un commit y un push, y por qué el repositorio es público desde el primer día (RNF-25).